

Alem do @Test

Organizacao, dados, dependencias e paralelismo em suites Java

O problema: de 1 teste para 10.000

- Executar um teste é simples. Organizar uma suite grande não é.
- Precisamos selecionar testes, alimentar dados e reduzir tempo.
- TestNG atua como framework/test runner de orquestração.

TestNG em uma frase

- Framework/test runner Java inspirado em JUnit e NUnit.
- Recursos: lifecycle, DataProvider, groups, dependencies e parallel.
- Versao usada: TestNG 7.12.0 com Java 21 + Maven.

Onde entra na piramide de testes?

- Unidade: Java + TestNG.
- Servico/API: TestNG + REST Assured ou clientes HTTP.
- UI/E2E: TestNG + Selenium/Appium.
- TestNG organiza a execucao; nao substitui essas ferramentas.

Nosso estudo de caso: TransferService

- Transferencia minima: R\$ 10.
- Transferencia maxima: R\$ 10.000.
- Valor nao pode ultrapassar o saldo.
- Transferencia valida reduz o saldo.

```
boolean isValid(balance, amount)  
amount >= 10  
amount <= 10000  
amount <= balance
```

@Test + ciclo de vida

- @Test identifica o caso executavel.
- @BeforeMethod prepara uma nova instancia antes de cada teste.
- @AfterMethod limpa o estado depois de cada teste.

```
@BeforeMethod  
setUp()  
  
@Test  
shouldAcceptAValidTransfer()  
  
@AfterMethod  
tearDown()
```

DataProvider + BVA: teoria virando execucao

- Fronteira inferior: 9, 10, 11.
- Fronteira superior: 9999, 10000, 10001.
- @DataProvider executa o mesmo teste para cada conjunto.
- BVA escolhe os valores; TestNG executa e verifica.

ECP: menos casos, mesma classe de comportamento

- Abaixo do minimo: -50 e 5 -> invalidos.
- Classe valida: 500 e 5000 -> validos.
- Acima do maximo: 15000 -> invalido.
- A tecnica define as classes; DataProvider fornece os dados.

Groups: execute apenas o que importa

- smoke: verificacoes essenciais e rapidas.
- regression: conjunto mais amplo.
- bva, ecp, workflow e parallel-demo: grupos didaticos.
- Exemplo: `mvn -Dgroups=smoke test`

Dependencies: fluxo controlado

- `authenticate()` -> `executeTransfer()` -> `logout()`.
- `dependsOnMethods` declara a dependencia entre cenarios.
- Se a dependencia falha, o teste dependente pode ser ignorado.
- Cuidado: excesso de dependencias aumenta acoplamento.

Paralelismo + testng.xml

- `parallel="methods"` distribui metodos por threads.
- `thread-count="3"` limita a tres threads neste exemplo.
- Cada cenario imprime a thread usada no terminal.
- Risco: estado compartilhado pode gerar race conditions.

```
<suite  
  parallel="methods"  
  thread-count="3">
```

Demo ao vivo: 5 minutos

- 1. Mostrar TransferService e um @Test.
- 2. Mostrar DataProvider com BVA/ECP.
- 3. Mostrar groups e dependsOnMethods.
- 4. Executar: mvn test
- 5. Executar: mvn -Psuite test e observar as threads.

TestNG x JUnit moderno

- Ambos: unit tests, assertions, parametrizacao e paralelismo.
- TestNG: DataProvider, groups, dependencies e testng.xml muito explicitos.
- JUnit: @ParameterizedTest, @Tag e ecossistema consolidado.
- Escolha depende da suite e do padrao da equipe.

Vantagens, limites e uso real

- Vantagens: dados, grupos, dependencias, suites e paralelismo.
- Cuidados: configuracao, acoplamento e thread safety.
- Uso verificavel: Apache Atlas e Apache BifroMQ usam TestNG.
- Adote quando esses recursos resolvem problemas reais.

Conclusao

- BVA e ECP definem O QUE testar.
- TestNG organiza COMO fornecer, agrupar, executar e avaliar.
- O diferencial aparece quando a suite precisa de orquestracao.

TestNG ganha valor quando a suite cresce.